# IntentFlow PoC-Lite Feasibility Report

Evidence that a changed authorization guard can be traced to approved product intent and reported as a PR-time verdict

| | |
|---|---|
| **Execution date** | 2026-09-08 |
| **Scope** | Local controlled Python fixture |
| **Outcome** | Unsafe and safe controls classified as expected |
| **Boundary** | Feasibility evidence, not production accuracy |

## Highest-risk assumption

Can IntentFlow recover structural context and approved intent from a code change, then produce an evidence-backed verdict instead of a generic review comment?

# Defense PoC-Lite package

The defense package combines a clickable high-fidelity prototype, a saved GitHub-like event and structured intent data check, a runnable local structural-analysis baseline, and a safe/unsafe risk experiment. The prototype is in outputs/08_clickable_prototype. It demonstrates the proposed workflow but does not claim that every interface page has a production service behind it.

# PoC boundary and method

The vertical slice uses Python ast because Tree-sitter was unavailable in the bundled runtime. A small adjacency graph replaces Neo4j, a saved event replaces a live GitHub webhook, and deterministic logic replaces optional model reasoning. These fallbacks validate the pipeline without claiming production integration or scale.

## Approved intent

REQ-001: Only an ADMIN can approve a pending refund.

## Evidence path

```
REQ-001 -> refund_service -> approve_refund -> RefundRepository.mark_approved
```

The analyzer compares the base and PR syntax trees, identifies changed functions, and checks whether an ADMIN role guard remains reachable before the protected call. It follows a local helper call in the safe refactor.

## Decision rule

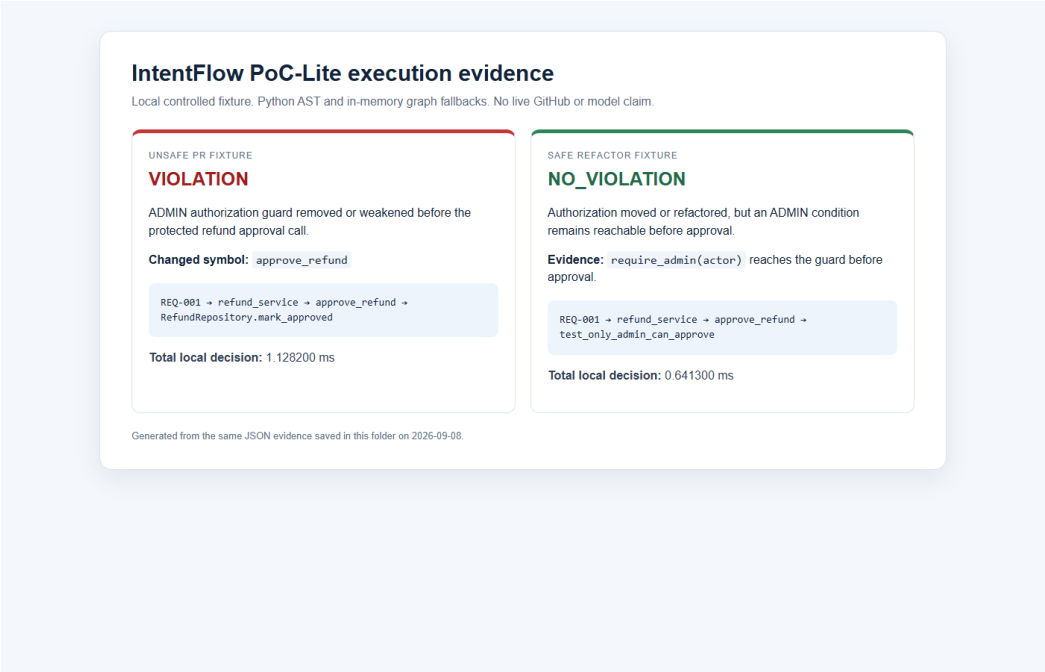
A removed effective ADMIN guard produces VIOLATION. A direct or helper-based guard that still executes before approval produces NO_VIOLATION.

# Actual execution results

| Case | Structural evidence | Verdict | Total ms |
|---|---|---|---|
| Unsafe PR | Base guard present; PR guard absent | **VIOLATION** | 1.128200 |
| Safe refactor | Helper keeps ADMIN guard before approval | **NO_VIOLATION** | 0.641300 |

Three unit tests passed: unsafe verdict, safe helper recognition, and trace reachability. The timings above come from the saved JSON evidence. They are local feasibility measurements on tiny fixtures.

# Rendered execution evidence




This browser-rendered image was generated from the same verdict JSON and timing files saved in the evidence folder. It clearly labels the local fallbacks and the absence of a live GitHub or model claim.

# GitHub-like output

The unsafe case generates a Check Runs style payload with conclusion failure and a changed-line annotation. The safe case generates conclusion success. Both JSON files state that the result was generated locally and was not published to GitHub.

## What the PoC supports

The run demonstrates a working path from approved intent to changed structural evidence, trace retrieval, deterministic decision and developer-facing evidence payload.

## What it does not support

The run does not establish final accuracy, TypeScript coverage, Tree-sitter behavior, Neo4j performance, large-repository scalability, live GitHub deployment, LLM semantic quality, merge-collision accuracy or predictive test-ranking performance.

## Reproduction

From outputs/07_poc_lite, run ./run_demo.ps1. The analyzer and tests use only the Python standard library. All machine-readable evidence remains under evidence/.

## Clickable interface

Run outputs/08_clickable_prototype/run_prototype.ps1. The interface uses local sample data and makes no live GitHub, Neo4j or external-model claim. Merge-collision and test-selection pages are interface demonstrations, not executed analytical results.